Zenith Optimizer

Evidencias de avance — Fase 2

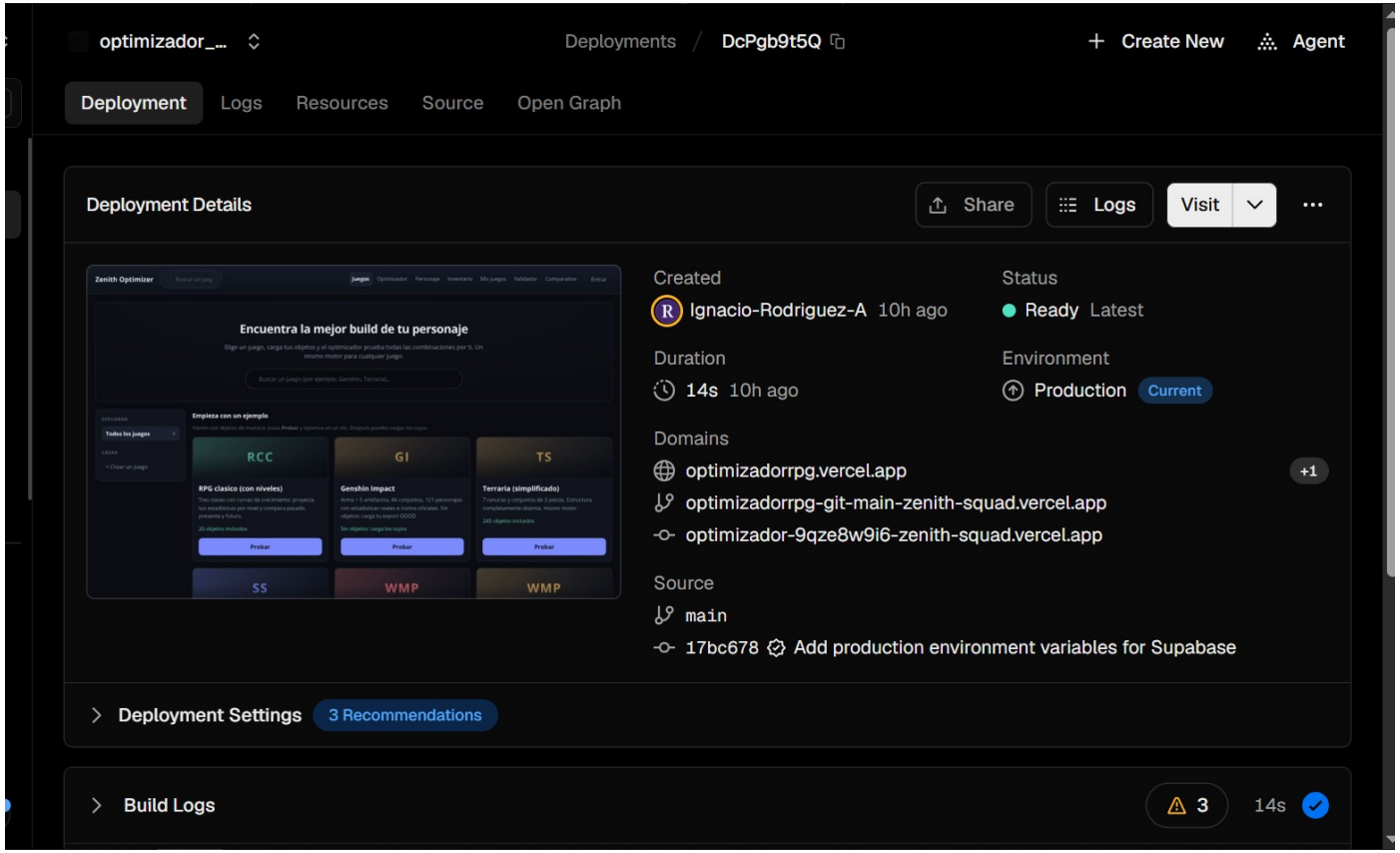
Asignatura	**Capstone PTY4614-002D — Duoc UC, sede Plaza Vespucio**
Equipo	**Zenith Squad — Ignacio Rodríguez, Arturo Moya, Matías Molina**
Aplicación	**optimizadorrpg.vercel.app**
Repositorio	**github.com/Cooounterfeit/Optimizador_RPG**
Fecha	**13 de octubre de 2026**

Las capturas de este documento se tomaron sobre el despliegue de producción. Cada evidencia indica qué demuestra y cómo puede verificarse de forma independiente.

EVIDENCIA 1 — DESPLIEGUE

La aplicación está en producción y es pública

Despliegue en Vercel con estado **Ready** y entorno **Production**, construido desde la rama **main** a partir del commit **17bc678**. El dominio público es **optimizadorrpg.vercel.app**; los otros dos son internos del equipo. Cada cambio en el repositorio dispara una compilación nueva, de modo que lo publicado corresponde siempre al código versionado y no a una copia manual.



EVIDENCIA 2 — CARÁCTER AGNÓSTICO

Un mismo motor para cuatro juegos distintos

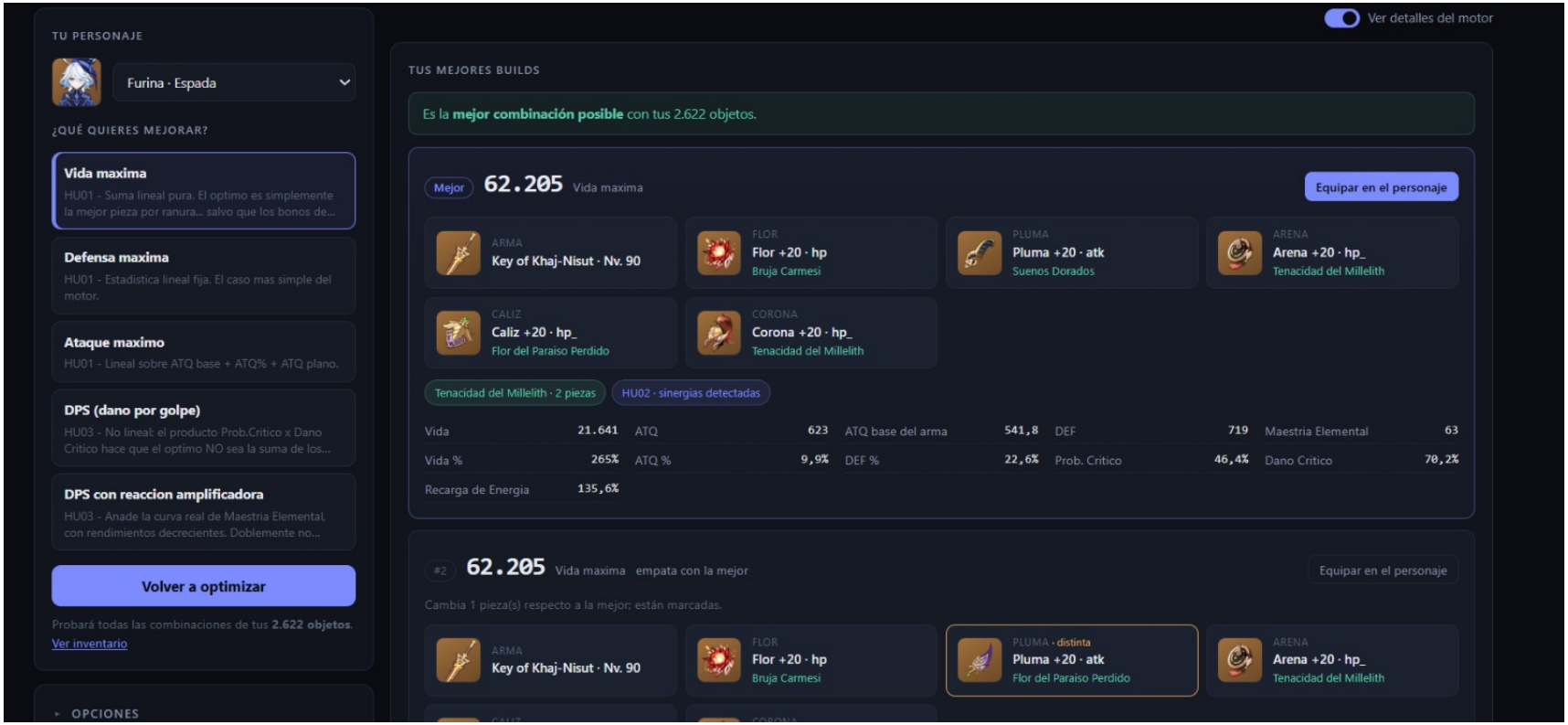
El catálogo del usuario con cuatro juegos cargados: Genshin Impact, dos especializaciones de WoW Midnight y un RPG clásico con curvas de crecimiento. Ninguno está escrito en el código del motor: cada uno es un archivo de configuración con sus estadísticas, ranuras, conjuntos y fórmulas. Esta pantalla es la evidencia directa del objetivo general del proyecto: añadir un juego no requiere modificar el programa.



EVIDENCIA 3 — MOTOR DE OPTIMIZACIÓN

Búsqueda exhaustiva sobre 2.622 objetos con óptimo demostrado

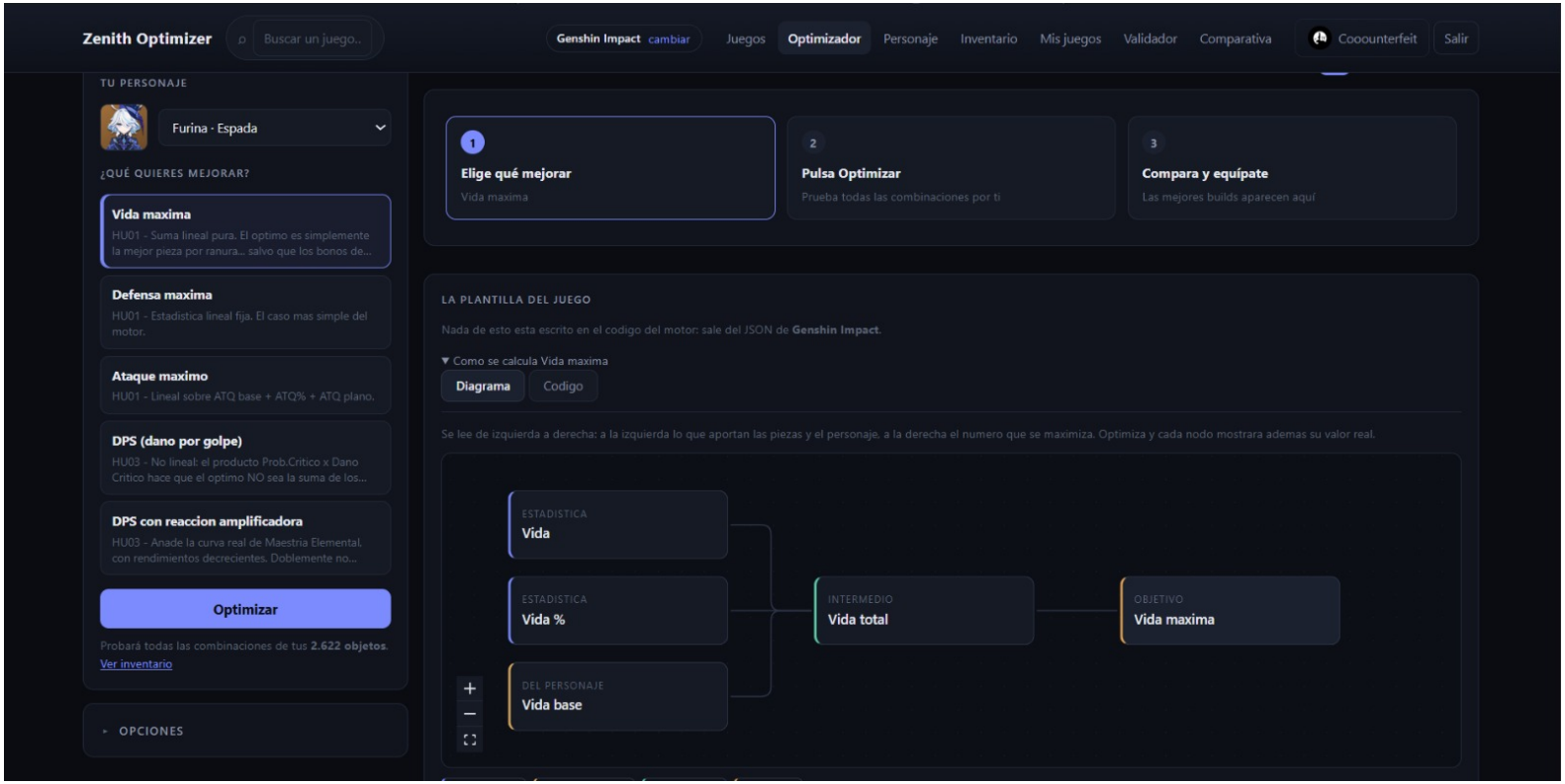
Resultado de optimizar Vida máxima para el personaje Furina. El motor declara **«es la mejor combinación posible con tus 2.622 objetos»**: no es una heurística ni una aproximación, sino ramificación y poda exacta, que descarta ramas solo cuando puede probar que no contienen el óptimo. Se muestran la build ganadora pieza por pieza, el conjunto activo (Tenacidad del Millelith, 2 piezas), las sinergias detectadas y el desglose completo de estadísticas. La segunda build empata en puntaje y se marca qué pieza cambia respecto de la primera.



EVIDENCIA 4 — TRAZABILIDAD DEL CÁLCULO

El usuario puede auditar de dónde sale cada número

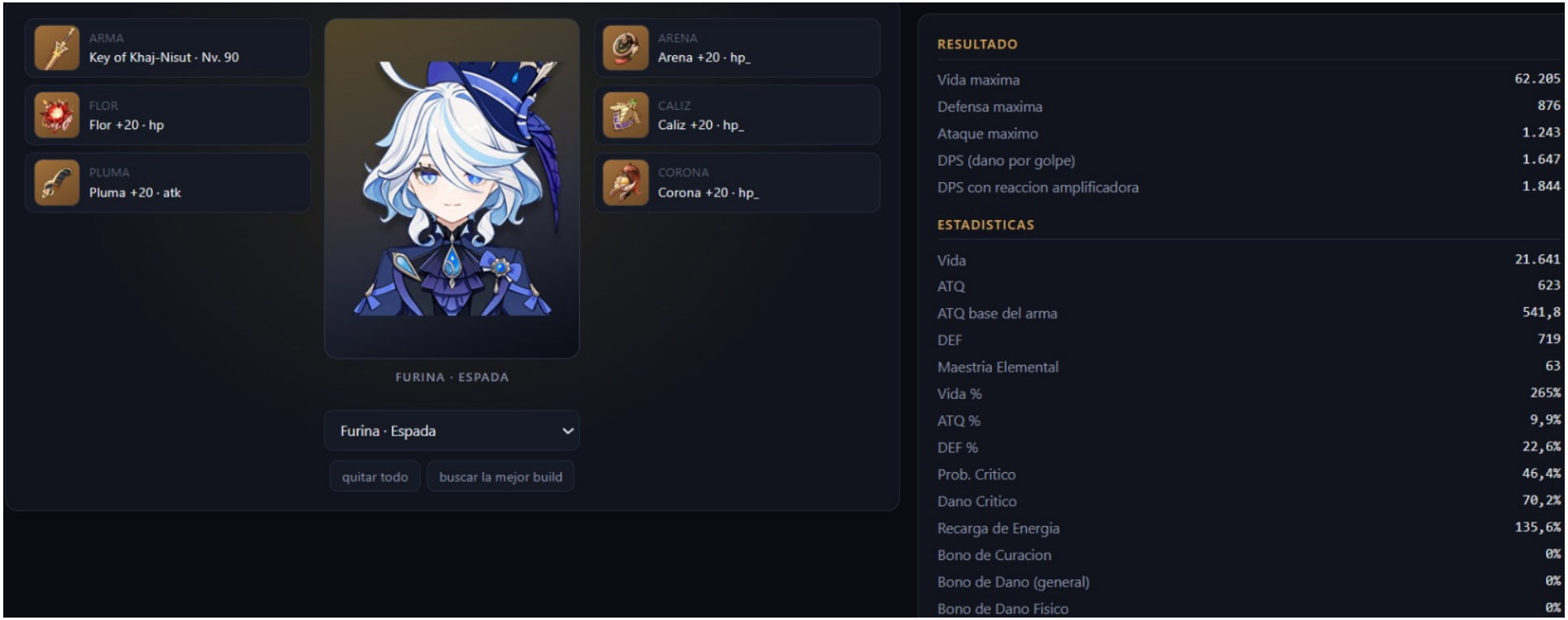
Grafo de dependencias del objetivo «Vida máxima», generado automáticamente a partir de la plantilla del juego. Se lee de izquierda a derecha: las estadísticas que aportan las piezas y el personaje alimentan un valor intermedio, y de ahí sale el objetivo que se maximiza. El propio sistema advierte que **«nada de esto está escrito en el código del motor: sale del JSON»**. La vista resuelve un problema real de las herramientas de este tipo: normalmente el cálculo es una caja negra.



EVIDENCIA 5 — FICHA DEL PERSONAJE

Los cinco objetivos del juego evaluados sobre la misma build

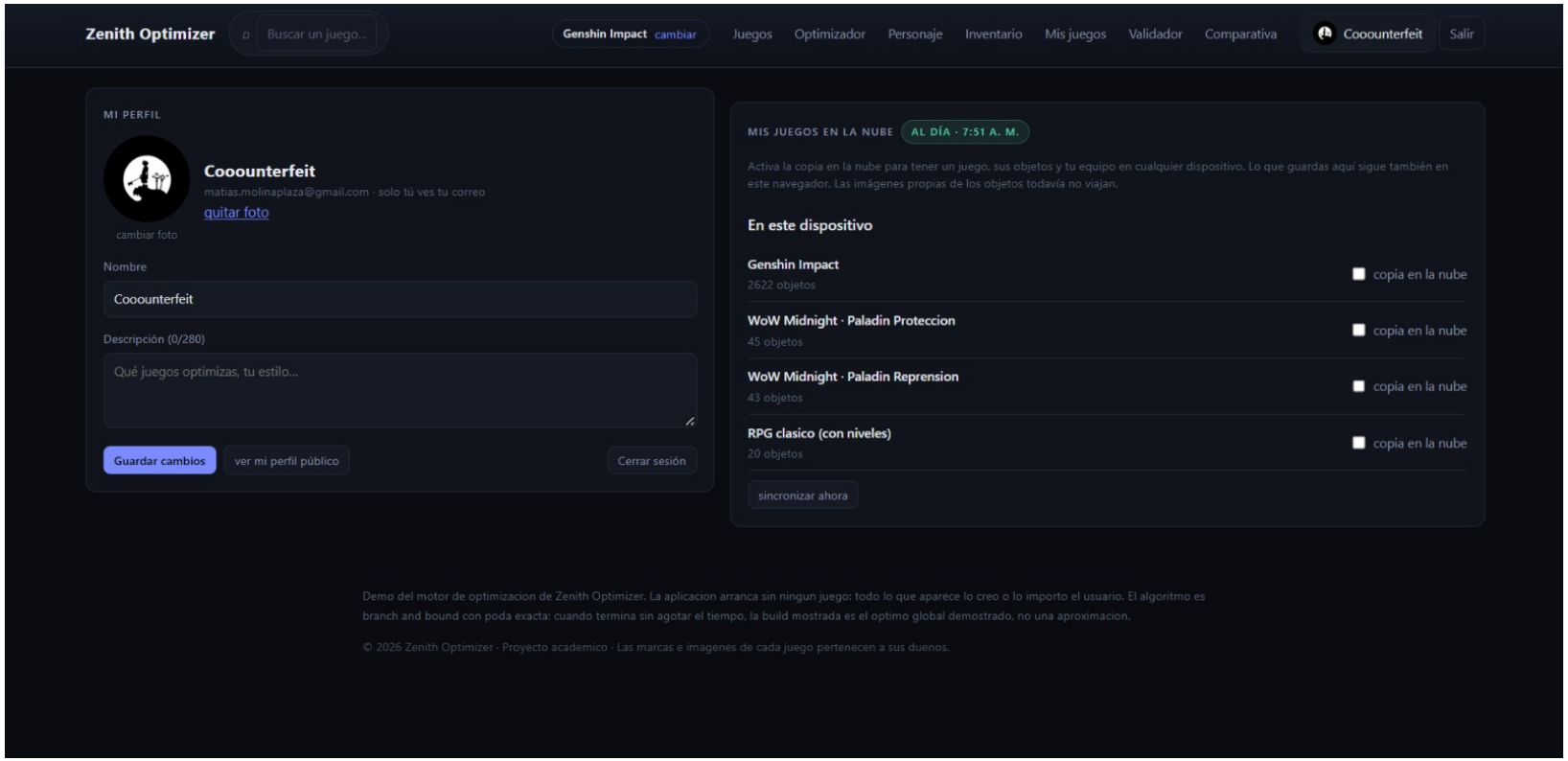
Equipo resultante y su lectura completa. A la derecha, los cinco objetivos declarados en la plantilla calculados simultáneamente —Vida, Defensa, Ataque, DPS por golpe y DPS con reacción amplificadora— junto al detalle de estadísticas. Los dos últimos objetivos no son lineales: el producto entre probabilidad y daño crítico hace que el óptimo no sea la suma de las mejores piezas por ranura, que es precisamente el caso que justifica un solver y no una simple ordenación.



EVIDENCIA 6 — CUENTAS Y SINCRONIZACIÓN

Registro, perfil y copia en la nube por juego

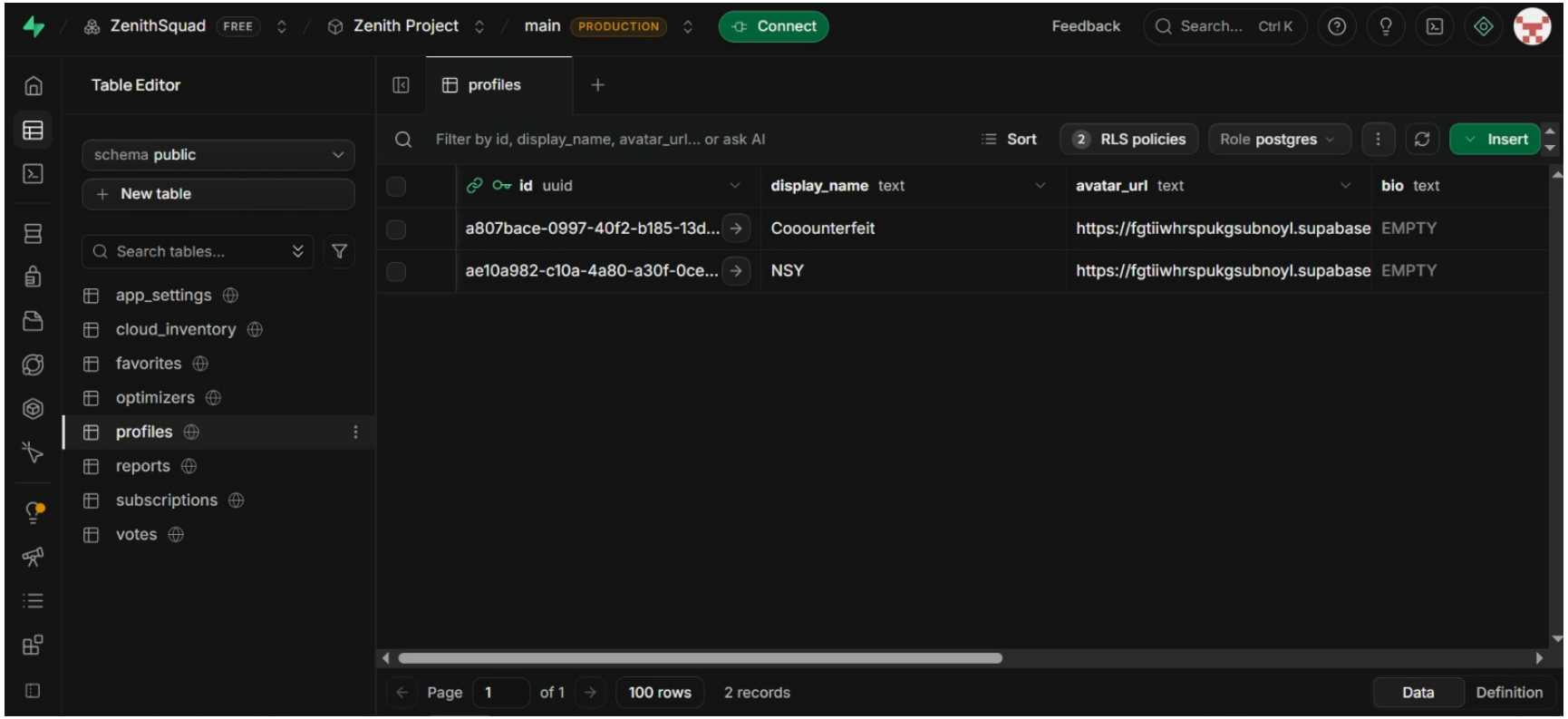
Perfil de un usuario autenticado. El sistema de cuentas está operativo de extremo a extremo: registro con confirmación por correo, inicio de sesión, foto de perfil y edición de datos. La copia en la nube se activa por juego, no de forma global, y la aplicación advierte explícitamente qué viaja y qué no. El correo queda visible solo para su dueño.



EVIDENCIA 7 — MODELO DE DATOS

Esquema relacional desplegado con Row Level Security

Editor de tablas de Supabase sobre la base de producción. Las ocho tablas del modelo están creadas —**app_settings, cloud_inventory, favorites, optimizers, profiles, reports, subscriptions, votes**— y la tabla **profiles** muestra dos registros reales con sus políticas de seguridad activas. El esquema se aplica desde archivos de migración versionados en el repositorio, no a mano, y su corrección se verifica con una batería de pruebas automatizadas que ejercita cada política con usuarios distintos.



CONTRASTE CON UNA HERRAMIENTA DE REFERENCIA

Validación cruzada contra Genshin Optimizer

Para comprobar la exactitud del motor se comparó el mismo personaje, con el mismo equipo, contra **Genshin Optimizer**, la herramienta de referencia de esa comunidad. Ambas calculan la Vida máxima de Furina sobre la misma build. Los valores no coinciden, y la diferencia resultó ser diagnóstica.

Concepto	Valor	Lectura
Genshin Optimizer (referencia)	**65.269 HP**	—
Zenith Optimizer	**62.205 HP**	—
Diferencia	**3.064 HP**	4,69 % del total — **20,02 % de la vida base** (15.307)

Qué descarta el análisis

La diferencia **no es un problema de redondeo**. El redondeo de los porcentajes que el juego muestra con un decimal puede desviar el total unos 38 HP en el peor caso; lo observado es ochenta veces más. Se probaron las hipótesis alternativas —omitir la subestadística del arma, omitir la vida plana de los artefactos— y ninguna reproduce el número: fallan por miles de puntos. La única que encaja, y encaja dentro del margen de redondeo, es la ausencia de un bono del 20 % sobre la vida base.

Causa identificada

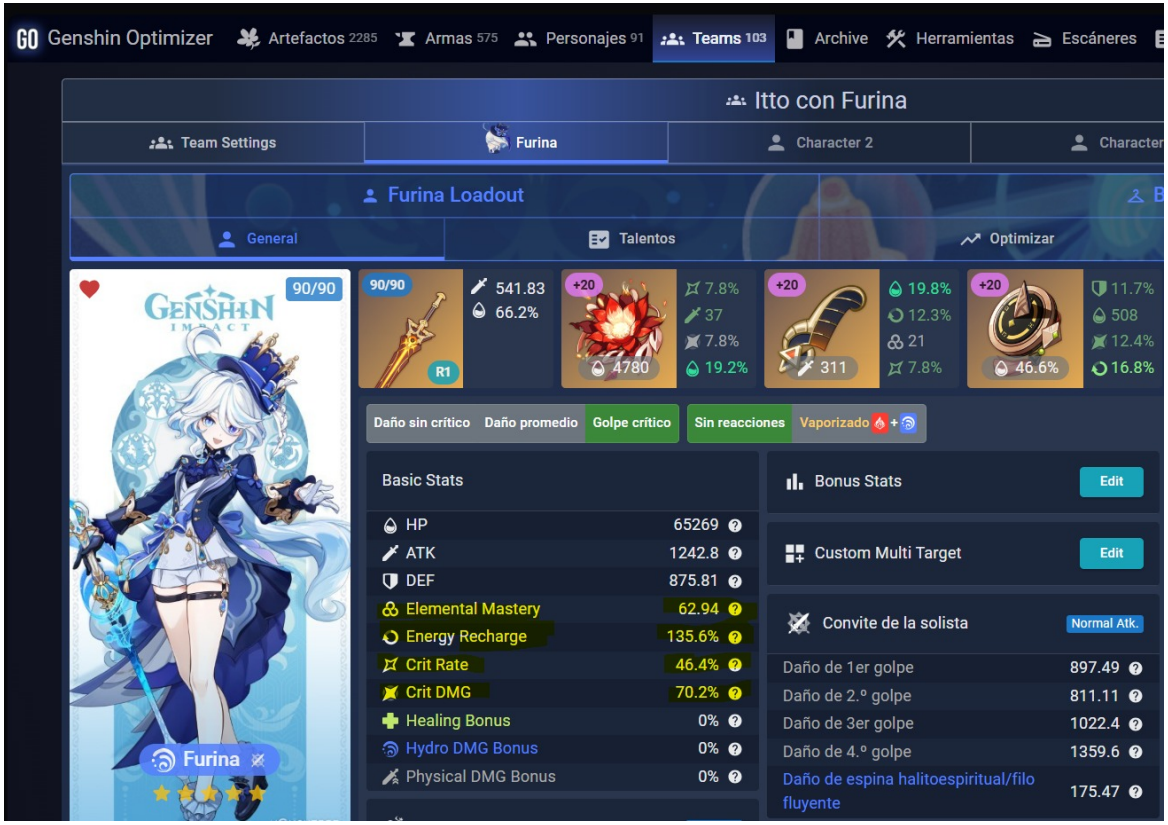
El bono ausente es el **pasivo del arma** (Key of Khaj-Nisut, +20 % de Vida). El adaptador que importa el inventario calcula el ataque base y la subestadística del arma, pero no sus efectos pasivos: lee el campo de refinamiento del archivo de origen y no lo utiliza, siendo que los pasivos escalan con él. En cambio, el bono de conjunto **sí** está correctamente modelado: Tenacidad del Millelith figura en la plantilla con su tramo de 2 piezas y +20 % de Vida, y el motor cuenta las piezas equipadas e incorpora los conjuntos a la cota de poda.

Alcance de la corrección. Se trata de una capacidad pendiente en el adaptador de importación, no de un defecto del motor ni del modelo de datos. La garantía de optimalidad no queda comprometida: el solver entrega el óptimo exacto del modelo que recibe; lo que falta es completar ese modelo con los efectos pasivos de las armas.

CONTRASTE — LADO A

Genshin Optimizer: 65.269 HP

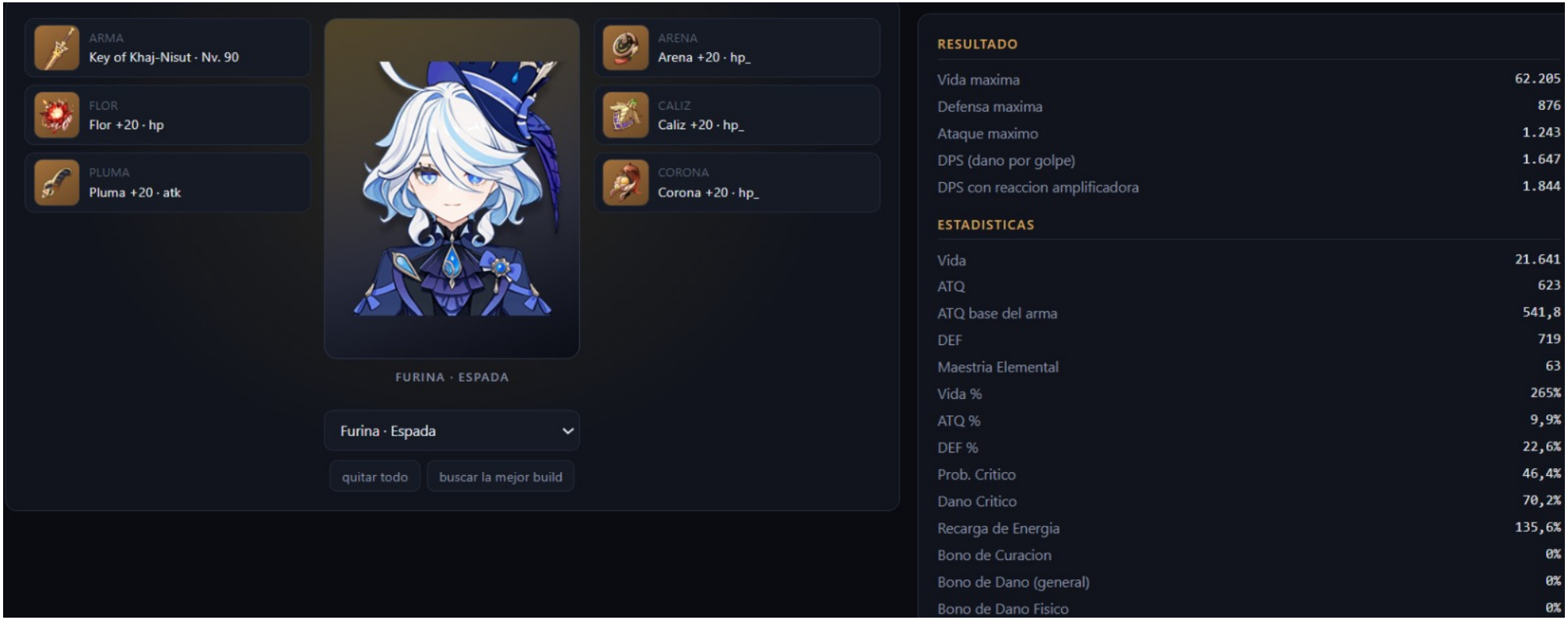
Herramienta de referencia de la comunidad. Mismo personaje (Furina), misma arma (Key of Khaj-Nisut, R1, Nv. 90) y mismas cinco piezas de artefacto.



CONTRASTE — LADO B

Zenith Optimizer: 62.205 HP

Mismo escenario en nuestro motor. La build elegida coincide pieza por pieza con la de referencia; lo que difiere es el total, por los 3.064 HP analizados en la página anterior.



SÍNTESIS

Qué acredita este conjunto de evidencias

Ámbito	Estado	Evidencia
Despliegue continuo en producción	Operativo	E1
Motor agnóstico validado con cuatro juegos	Operativo	E2
Optimización combinatoria con óptimo demostrado	Operativo	E3
Trazabilidad del cálculo para el usuario	Operativo	E4
Autenticación, perfiles y copia en la nube	Operativo	E6
Modelo de datos con Row Level Security	Operativo	E7
Efectos pasivos de armas en la importación	Pendiente, diagnosticado	Contraste

La comparación contra una herramienta de referencia no se incluye como incidencia sino como método: detectó una diferencia de 3.064 HP, permitió descartar el redondeo y las hipótesis alternativas por aritmética, y localizó la causa en una función concreta del adaptador de importación. Es el tipo de verificación que distingue un resultado comprobado de uno simplemente mostrado en pantalla.